专业：生物医学工程
姓名：贺子珏
学号：3230100633
日期：2026/03/26
地点：教六 - 204

浙江大学 实验报告

课程名称：硬件描述语言　指导老师：余锋　实验名称：HDL 实验一

装订线

一、实验要求

（1）流水灯

```
module stream_led(
    input              clk_i,
    input              rst_i,
    output reg [7:0]    led_o
);

    reg [10:0]  clk_cnt;

    // add code here

endmodule
```

1. 提供输入：

clk_i	时钟信号，100MHz
rst_i	复位信号，高电平复位

2. 要求输出：

led_o	初始状态为 led_o[0]亮，其它灯不亮，每过 10us 左右，亮的灯向左传递，当传递到最后的 led_o[7]时，下一次再传递为 led_o[0]

（2）数字秒表

```
module stopwatch(
    input              clk_i,
    input              rst_i,
    input              trig_i,
    output reg [10:0]   millisec_o,
    output reg [5:0]    second_o,
    output reg          valid_o
);
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

```
    reg [3:0]   base_cnt;
    reg [10:0]  millisec;
    reg [5:0]   second;

    // add code here

endmodule
```

1. 提供输入：

clk_i	时钟信号，100MHz
rst_i	复位信号，高电平复位
trig_i	触发信号，在该信号有效时输出一次计时数据

2. 要求输出：

second_o	“秒”输出
millisec_o	“毫秒”输出
valid_o	表示输出有效

3. 实验提示

（1）当 rst_i 信号有效时秒表的输出全为 0，无效后开始计时。

（2）该实验需要 3 个计数器，一个基本计数器 base_cnt，一个“毫秒”计数器 millisec，一个“秒”计数器 second。

（3）base_cnt 值每达到 8 则更新 millisec 的值加 1，同时 base_cnt 清零，millisec 值每达到 1024 则更新 second 的值加 1，同时 millisec 清零，second 值每达到 59 后重新开始计时。

（4）一旦 trig_i 信号有效则将当前 millisec 输出到 millisec_o，将当前 second 输出到 second_o，同时将 valid_o 置为高电平表示输出数据有效。

装订线

二、实验代码与注释

（1）流水灯

stream_led.v

```
module stream_led(
    input               clk_i,
    input               rst_i,
    output reg [7:0]     led_o
);

    reg [10:0]  clk_cnt;
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

装订线

```
    always @(posedge clk_i or posedge rst_i) begin
        // asynchronous reset: active high
        if (rst_i) begin
            // clear all regs
            clk_cnt <= 0;
            // set the output
            led_o <= 8'b0000_0001;
        end

        // stream led
        else begin
            // clk: 100MHz => 1T = 1 / 100MHz = 0.01us
            // Delay 10us: 10us = 1000 * 0.01us => clock counter counts up to 1000
            if (clk_cnt == 11'd999) begin
                // reset the clock counter
                clk_cnt <= 11'd0;
                // shift left circularly
                if (led_o[7] == 1'b1) begin
                    led_o <= 8'b0000_0001;
                end
                else begin
                    led_o <= led_o << 1;
                end
            end
            else begin
                clk_cnt <= clk_cnt + 11'd1;
            end
        end
    end

endmodule
```

tb_stream_led.v

```
`timescale 1ns/1ns

module tb_stream_led;

    reg        clk;
    reg        rst;
    wire [7:0]  led;

    stream_led inst_stream_led(
        .clk_i(clk),
        .rst_i(rst),
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

装订线

```
        .led_o(led)
    );

    // init basic signal
    initial begin
        // init clock
        clk <= 1'b0;
        // reset the module first
        rst <= 1'b1;
        #100;
        rst <= 1'b0;
    end

    // clock = 100MHz => T = 10ns
    always begin
        #5 clk = ~clk;
    end

    // output vcd file
    initial begin
        $dumpfile("tb_stream_led.vcd");
        $dumpvars;
        #1000000
        $finish;
    end

endmodule
```

（2）数字秒表

stopwatch.v

```
module stopwatch(
    input              clk_i,
    input              rst_i,
    input              trig_i,
    output reg [10:0]   millisec_o,
    output reg [5:0]    second_o,
    output reg          valid_o
);

    reg [3:0]   base_cnt;
    reg [10:0]  millisec;
    reg [5:0]   second;
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

装订线

```
always @(posedge clk_i or posedge rst_i) begin
    // asynchronous reset: active high
    if (rst_i) begin
        // clear all regs
        base_cnt <= 4'd0;
        millisec <= 11'd0;
        second <= 6'd0;
        // clear all output regs
        millisec_o <= 11'd0;
        second_o <= 6'd0;
        valid_o <= 1'b0;
    end

    // update timer regs
    else begin
        // default: valid_o = 0
        valid_o <= 1'b0;

        // basic counter: add to 8
        if (base_cnt == 4'd8) begin
            // reset base_cnt
            base_cnt <= 4'd0;

            // millisec counter: add to 1024
            if (millisec == 11'd1024) begin
                // reset millisec
                millisec <= 11'd0;

                // second counter: add to 59
                if (second == 6'd59) begin
                    second <= 6'd0;
                end
                else begin
                    second <= second + 6'd1;
                end
            end
            else begin
                millisec <= millisec + 11'd1;
            end
        end
        else begin
            base_cnt <= base_cnt + 4'd1;
        end
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

```
        end
    end

    // trigger signal
    always @(*) begin
        if (trig_i) begin
            millisec_o <= millisec;
            second_o <= second;
            valid_o <= 1'b1;
        end
        else begin
            valid_o <= 1'b0;
        end
    end

endmodule
```

编译的时候发现，寄存器变量 millisec_o, second_o 和 valid_o 同时出现在两个 always 块中，导致硬件电路上一个输出连了多根线。于是修改 trigger signal 部分代码，将 trig_i 触发部分放到时序电路内部。

另外，对于“base_cnt 值每达到 8 则更新 millisec 的值加 1，同时 base_cnt 清零，millisec 值每达到 1024 则更新 second 的值加 1”的理解有误，将 bas_cnt 和 millisec 的周期进行修改。

修改后，stopwatch.v 内容如下。

装

订

线

stopwatch.v

```
module stopwatch(
    input              clk_i,
    input              rst_i,
    input              trig_i,
    output reg [10:0]   millisec_o,
    output reg [5:0]    second_o,
    output reg          valid_o
);

    reg [3:0]   base_cnt;
    reg [10:0]  millisec;
    reg [5:0]   second;

    always @(posedge clk_i or posedge rst_i) begin
        // asynchronous reset: active high
        if (rst_i) begin
            // clear all regs
            base_cnt <= 4'd0;
            millisec <= 11'd0;
            second <= 6'd0;
            // clear all output regs
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

装订线

```
            millisec_o <= 11'd0;
            second_o <= 6'd0;
            valid_o <= 1'b0;
        end

        // update timer regs
        else begin
            // basic counter: T_cnt = 8
            if (base_cnt == 4'd7) begin
                // reset base_cnt
                base_cnt <= 4'd0;

                // millisec counter: T_millisec = 1024
                if (millisec == 11'd1023) begin
                    // reset millisec
                    millisec <= 11'd0;

                    // second counter: T_second = 60
                    if (second == 6'd59) begin
                        second <= 6'd0;
                    end
                    else begin
                        second <= second + 6'd1;
                    end
                end
                else begin
                    millisec <= millisec + 11'd1;
                end
            end
            else begin
                base_cnt <= base_cnt + 4'd1;
            end

            // trigger signal: output the time or not
            if (trig_i) begin
                millisec_o <= millisec;
                second_o <= second;
                valid_o <= 1'b1;
            end
            else begin
                valid_o <= 1'b0;
            end
        end
    end
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

```
endmodule
```

tb_stoopwatch.v

```
`timescale 1ns/1ns

module tb_stopwatch;

    reg        clk;
    reg        rst;
    reg        trig;
    wire [5:0]  second;
    wire [10:0] millisec;
    wire       valid;

    // instantiate the module: stopwatch
    stopwatch inst_stopwatch(
        .clk_i(clk),
        .rst_i(rst),
        .trig_i(trig),
        .millisec_o(millisec),
        .second_o(second),
        .valid_o(valid)
    );

    // init basic signal
    initial begin
        // init clock and trigger
        clk <= 1'b0;
        trig <= 1'b0;
        // reset the module first
        rst <= 1'b1;
        #100;
        rst <= 1'b0;
    end

    // clock = 100MHz => T = 10ns
    always begin
        #5 clk = ~clk;
    end

    // test the trigger
    initial begin
```

装订线

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

装订线

```
        // wait for the reset
        #200;

        // 1st: trig
        trig = 1'b1;
        #10;
        trig = 1'b0;

        // wait for 500 clk
        #5000;

        // 2nd: trig
        trig = 1'b1;
        #10;
        trig = 1'b0;

        // wait for 5000 clk
        #50000;
        trig = 1'b1;
        #10;
        trig = 1'b0;

        // finish
        #5000000;
        $display("[Finish]");
        $finish;
    end

    always @(posedge clk) begin
        if(valid)
            $display("%d second %d millisecond", second, millisec);
    end

    // output vcd file
    initial begin
        $dumpfile("tb_stopwatch.vcd");
        $dumpvars;
    end

endmodule
```

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

三、仿真结果与分析

（1）流水灯

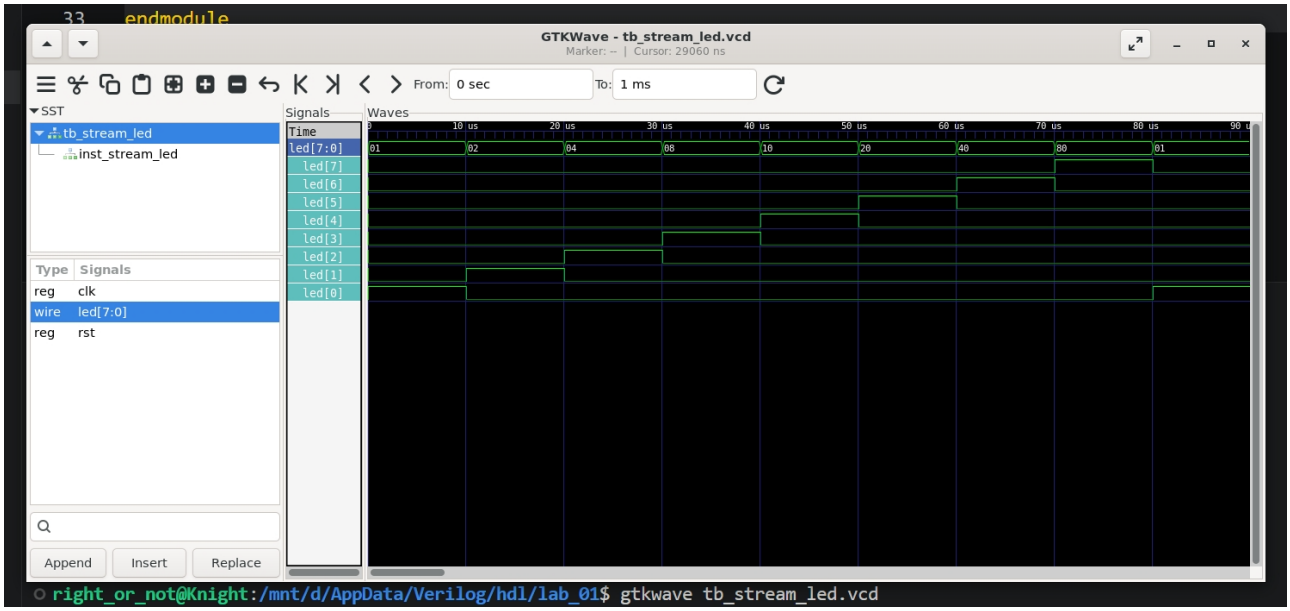
在 bash 中输入以下命令，通过 GTKWave 查看波形文件 tb_stream_led.vcd。

```
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01$ iverilog -o ./output/stream_led.vvp -y ./ tb_stream_led.v
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01$ cd output
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01/output$ vvp stream_led.vvp
VCD info: dumpfile tb_stream_led.vcd opened for output.
tb_stream_led.v:35: $finish called at 1000000 (1ns)
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01/output$ gtkwave tb_stream_led.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[1000000] end time.
```

选取需要查看的输出波形 led[7:0]，即可查看输出。



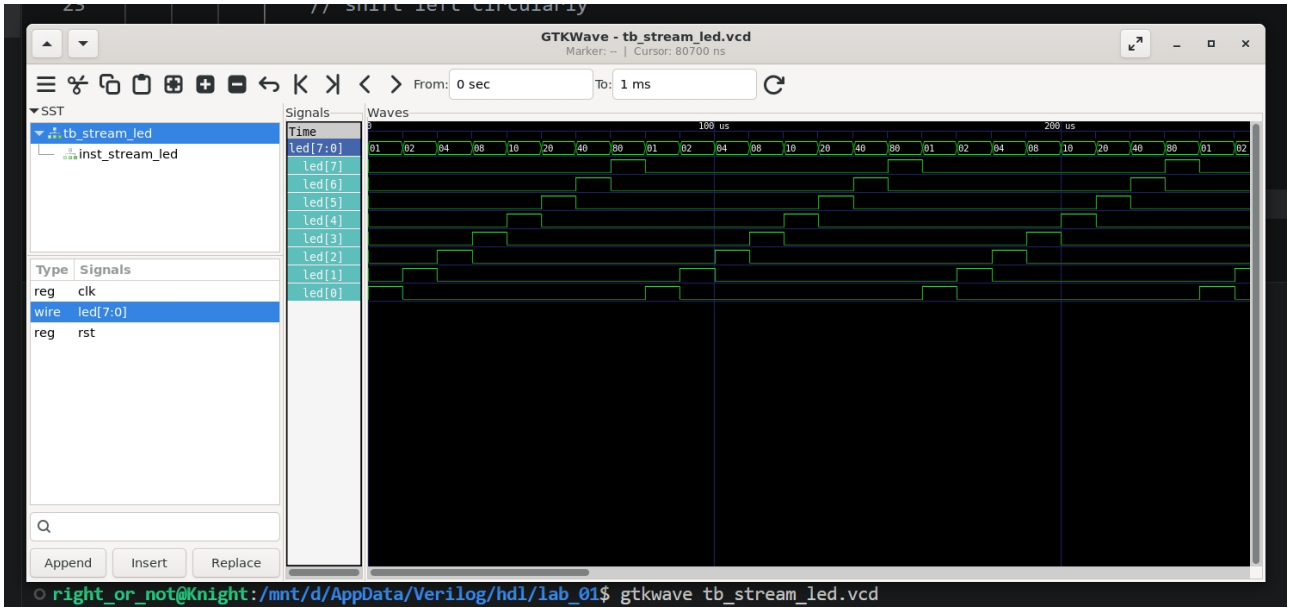


图 1　流水灯实验波形图

装订线

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

根据以上输出波形图 1，可以看到：初始状态 led[0]为高电平，led[0]电量，之后每过大约 10us，led 的信号向高位移动，实现了流水灯的效果。同时，当 led[7]被点亮后的下一个周期，又能够返回使得 led[0]被点亮。成功实现了流水灯的效果。

（2）数字秒表

在 bash 中输入以下命令，通过 GTKWave 查看波形文件 tb_stopwatch.vcd。

```
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01$ iverilog -o ./output/stopwatch.vvp -y ./ tb_stopwatch.v
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01$ cd output
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01/output$ vvp stopwatch.vvp
VCD info: dumpfile tb_stopwatch.vcd opened for output.
 0 second    1 millisecond
 0 second   56 millisecond
 5 second  487 millisecond
[Finish]
tb_stopwatch.v:65: $finish called at 10505230 (1ns)
right_or_not@Knight:/mnt/d/AppData/Verilog/hdl/lab_01/output$ gtkwave tb_stopwatch.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[10505230] end time.
```

选取需要输出观察的信号：

1. clk_i
2. rst_i
3. trig_i
4. base_cnt[3:0]
5. millisec[10:0]
6. second[5:0]
7. mllisec_o[10:0]
8. second_o[5:0]
9. valid_o

观察时，将输出的 Data Format 改为十进制会有利于对模块功能实现的验证。

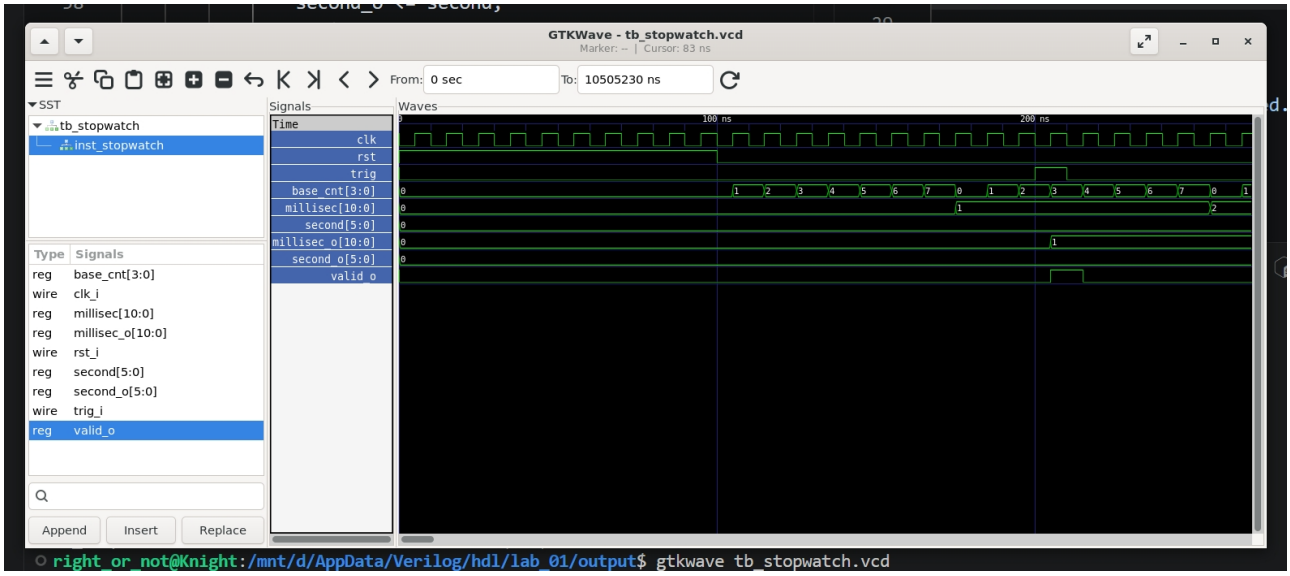


图 2.1　数字秒表实验波形图 － 触发信号 1

从波形图 2.1, 2.2, 2.3, 2.4 中可以看出：

1. clk 信号以 10ns 为周期发出周期时钟信号，频率为 100MHz。
2. 在前 100us，rst 信号为高电平，模块执行复位程序，所有内部寄存器和输出均强制清零。
3. 在 trig 给出高电平信号的一个周期内，遇到时钟的上升沿，valid_o 就能够实现输出当前时间的功

装订线

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

能（millisec_o 得到 millisec 的值，second_o 得到 second 的值，并且 valid_o 输出 1 表示输出有效）。

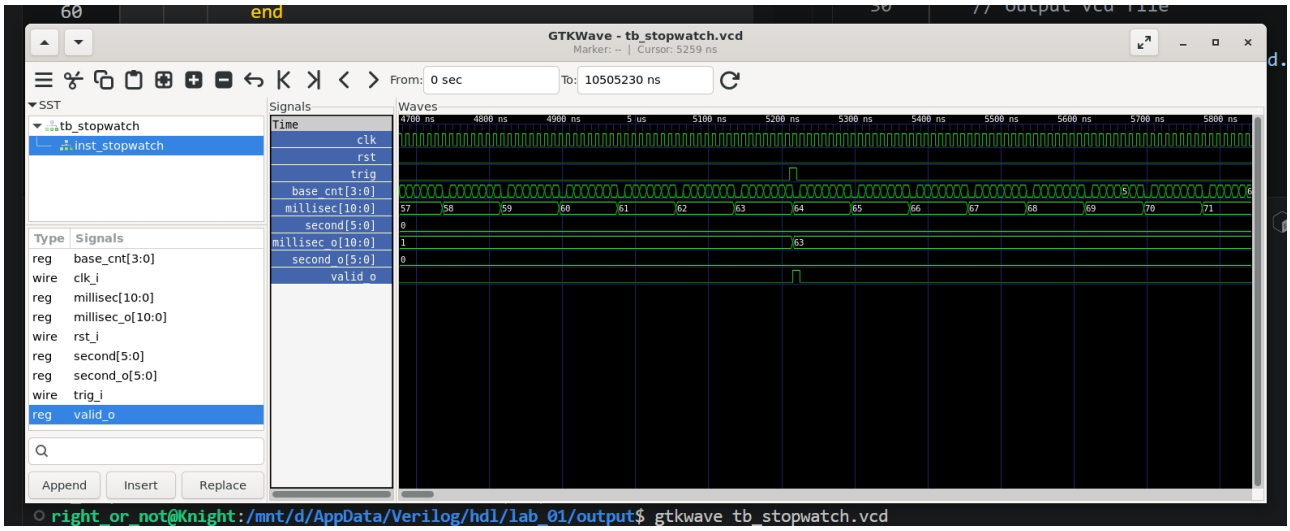


图 2.2　数字秒表实验波形图 － 触发信号 2

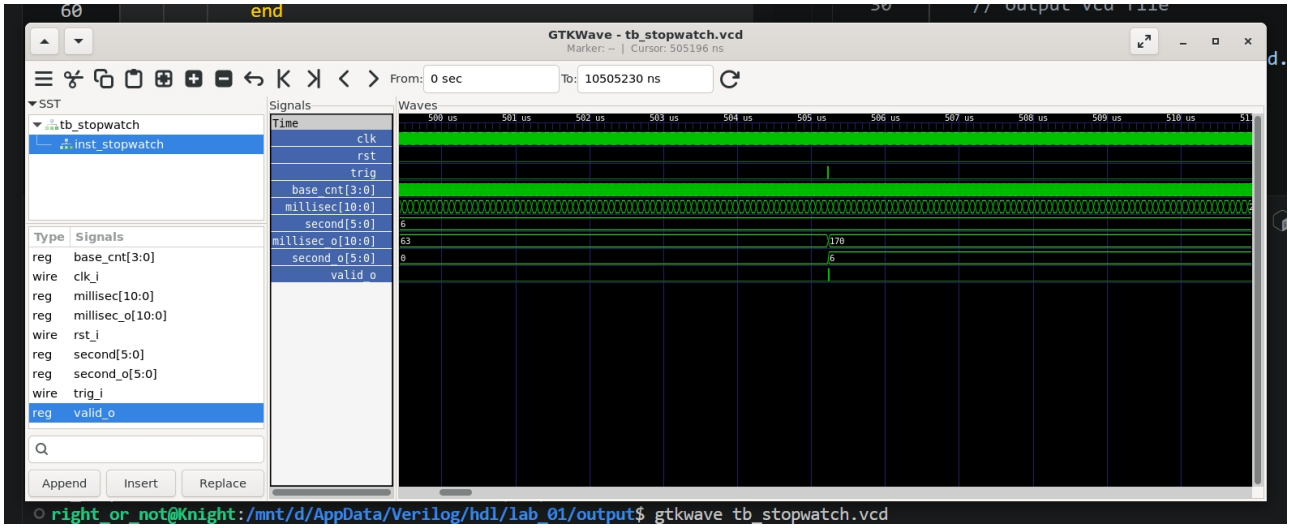


图 2.3　数字秒表实验波形图 － 触发信号 3

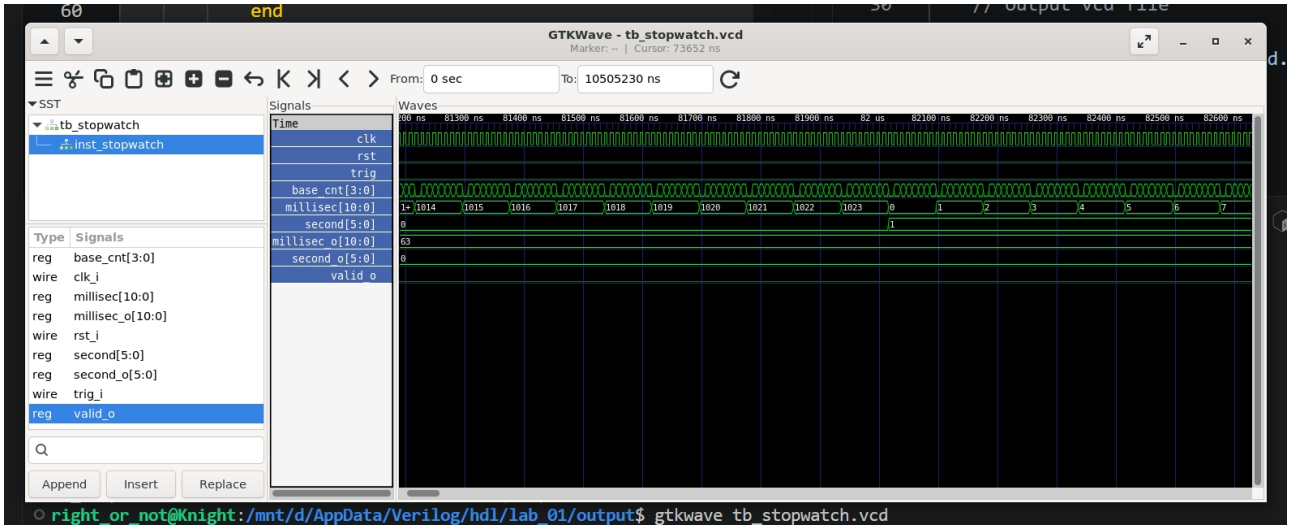


图 2.4　数字秒表实验波形图 － millisec 达到一个周期清零

装

订

线

实验名称：HDL 实验一　姓名：贺子珏　学号：3230100633

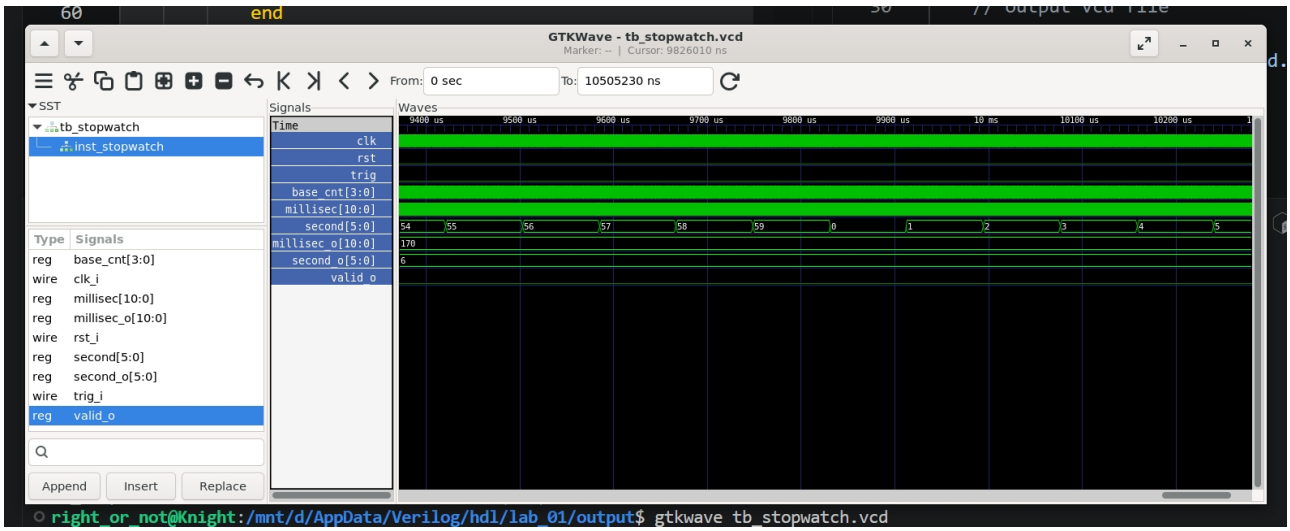


图 2.5　数字秒表实验波形图 – second 达到一个周期清零

4. base_cnt 在达到 8 的时候，马上让 millisec 加 1，并给自己清零；同样，millisec 和 second 的循环计数功能也得到了实现（millisec 的周期为 1024，second 的周期为 60）。

装

订

线

四、讨论与心得

在本次实验中，我通过实验学习了 Verilog 代码的基本结构，由于仿真软件需要路径没有中文，而笔记本的系统用户名为中文，因此采用了 iverilog + GTKWave 的组合实现 Verilog 的仿真。（都怪可恶的 Windows 系统!!! Linux 是对的!!!）

在实验中，我切实感受到 Verilog 的代码形式和 C 语言很像，但硬件描述语言与普通的编程语言相比，还是存在许多不同之处。对于较为陌生的 Verilog，VSCode 的代码补全功能给了我很大的帮助。在使用 iverilog 实现编译和仿真的时候，出现了很多问题，实验中通过 98 上的各种经验贴、查阅 CSDN 上的经验贴以及询问 AI，最后实现了仿真。在仿真过程中，遇到了很多硬件描述语言与传统 C 语言的差异，比如触发逻辑等问题、一个输出不能在不同 always 块中被幅值等等，这些规则都与实际电路的连线相对应，需要在实验中不断练习、习惯。

另外，实验中也有很多细节问题供我思考，例如对实验要求细节的理解，“base_cnt 值每达到 8 则更新 millisec 的值加 1”表达的是 base_cnt 的周期是 8，还是在 base_cnt 等于 8 后的下一个周期再执行 millisec 加 1 的操作？诸如此类的细节问题也需要在实验中不断深化理解，实现与目标相符的电路。

在掌握 Verilog 的设计要点和设计范式之前，还需要大量的学习和实践。